

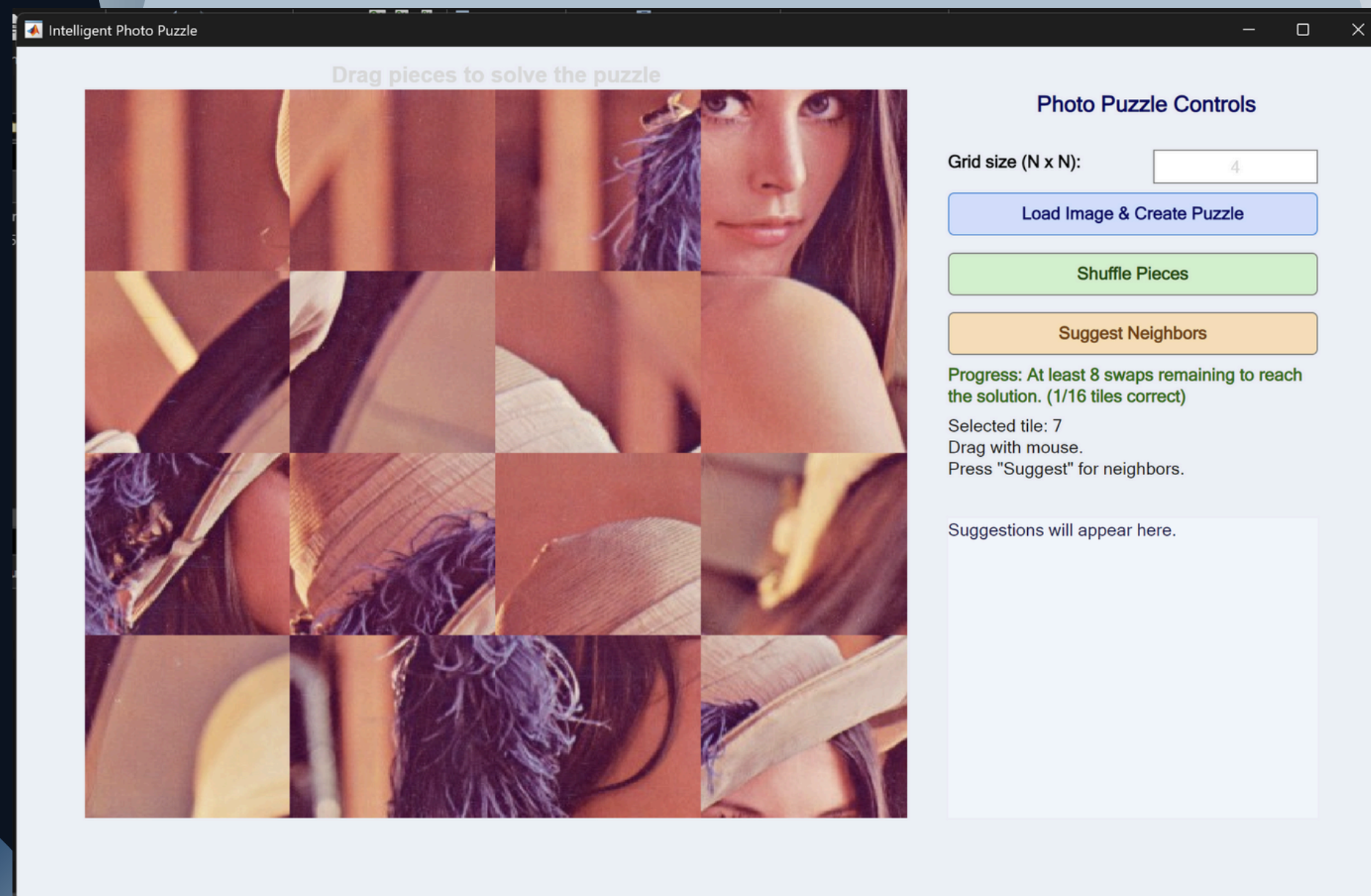
Group 3

INTELLIGENT PHOTO-PUZZLE

COMPUTER VISION PROJECT

Presented by

Jyotsana Parkhedkar
Harshini Narra
Rishikesh Sirisilla
Prathamesh Kale



OVERVIEW

- **Goal:** Build an interactive Photo Puzzle GUI in MATLAB.

Features:

- Load any image
- Split into $N \times N$ tiles
- Drag-and-drop gameplay
- Intelligent neighbor suggestions using CV algorithms

Algorithms used:

- Edge matching
- Color histogram similarity
- SURF feature matching



PROJECT MOTIVATION

Motivation

- Jigsaw puzzle solving is a classic image matching and image understanding task.
- Real-world relevance: document reconstruction, texture stitching, image mosaicing, robotics assembly.

Course relevancy

- Uses image filtering & edges → low-level vision topics
- Uses feature matching (SURF) → mid/high-level vision topics (feature detection, object matching).
- Uses color, perception, image segmentation concepts

PROBLEM STATEMENT

Problem

Problem

- Given an image split into $N \times N$ tiles:
 - a. Shuffle the tiles randomly.
 - b. Allow the user to interactively rearrange tiles.
 - c. Provide AI-assisted suggestions to help find correct neighbors using image similarity metrics.

KEY CHALLENGES

- Representing tiles and grid positions
- Efficient drag-and-drop interaction
- Computing similarity between tiles robustly
- Combining multiple matching cues into one suggestion score

SURVEY

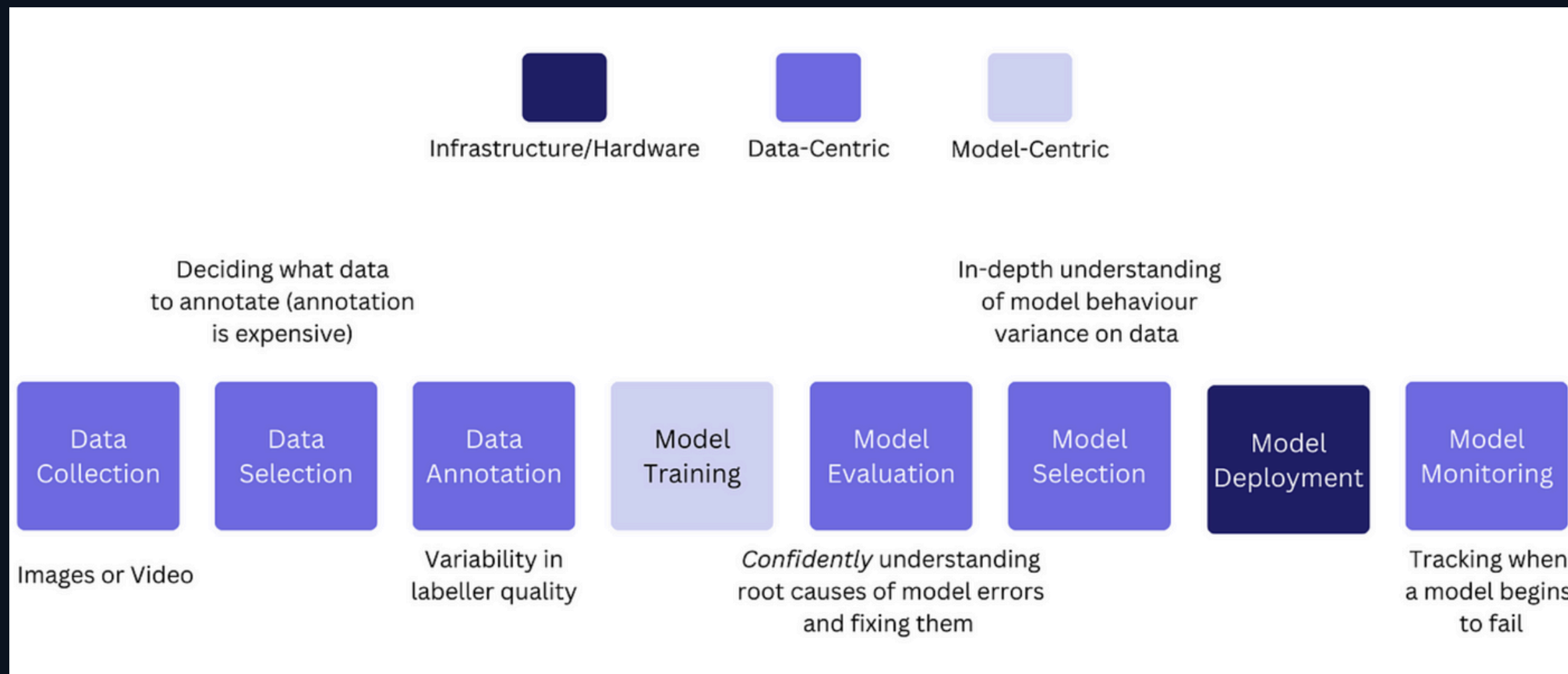
EXISTING APPROACHES

- Manual puzzle solving → based on human perception of edges, colors, patterns.
- Basic digital puzzles → allow dragging pieces but no intelligence.

STATE-OF-THE-ART RESEARCH

- Jigsaw puzzle solvers use:
 - Edge compatibility metrics
 - Color histogram similarity
 - Keypoint matching (SIFT/SURF)
 - Graph-based global optimization
- Used in: mosaic reconstruction, archaeological restoration, shredded document reconstruction.

MAIN IDEA



Tile Generation

- Resize input image
- Cut into $N \times N$ uniform patches

Shuffling

- Fisher–Yates shuffle (MATLAB randperm)

Feature Extraction per Tile

- Edge descriptor: concatenation of border pixels
- Color histogram (RGB): 16 bins/channel
- SURF keypoints + descriptors: local texture patterns

Similarity Computation

- Edge distance $\rightarrow$ L2 norm
- Color similarity $\rightarrow 1 - \text{cosine similarity}$
- SURF similarity $\rightarrow$ number of matched keypoints

Combined ranking

- Normalize scores
- Combined score = edge + color – SURF
- Top-3 tiles selected as suggestions

EXPERIMENTAL SETUP

- **Dataset:**

- Used user-selected images (natural scenes, objects, faces).
- Tested on multiple sizes: 3×3 , 4×4 , 5×5 .

- **Baseline Implementation:**

- Without intelligence $\rightarrow$ user purely drags tiles.
- Naïve similarity measure: only edge matching.

- **Performance Metrics:**

- Suggestion accuracy: how often suggested tiles are actually neighbors.
- User effort: number of drag attempts needed.
- Runtime: time to compute features and suggestions.



EXPERIMENTAL RESULTS

Baseline observations

- Using only edges gives poor results when edges have uniform intensity.

Our improved version

- Adding color histograms increases accuracy significantly for natural images.
- Adding SURF matching helps detect tiles with strong textures (wood, leaves, patterns).
- Combined score consistently provides:
 - More meaningful suggestions
 - Higher match accuracy
 - Lower user interaction overhead



Drag pieces to solve the puzzle



Photo Puzzle Controls

Grid size (N x N):

4

Load Image & Create Puzzle

Shuffle Pieces

Suggest Neighbors

Progress: At least 8 swaps remaining to reach the solution. (1/16 tiles correct)

Selected tile: 7

Drag with mouse.

Press "Suggest" for neighbors.

Suggestions will appear here.

REFERENCES

SURF Features

- Bay et al., "Speeded-Up Robust Features (SURF)," 2008.

Jigsaw puzzle solving in computer vision

- Cho et al., "A probabilistic model for image jigsaw puzzles," CVPR 2010.
- Gallagher, "Jigsaw puzzles with pieces of unknown orientation," CVPR 2012.

Color & edge compatibility metrics

- Pomeranz et al., "Fully automatic puzzle solving using computer vision," CVPR 2011.

Course Textbook Connections

- Gonzalez & Woods, Digital Image Processing (Chapters: filtering, edge detection, feature matching).

SUMMARY

WHAT WE LEARNED

- How to extract and compare features at low, mid, and high levels of vision.
- Importance of combining multiple cues (edges + color + texture).
- How GUI design supports interactive CV applications.

Tips for other teams

- Always normalize similarity scores before combining.
- Start with simple matching, then add advanced features.

FUTURE WORK

- Use deep learning patch embeddings (CNN features).
- Add orientation-invariant matching.
- Implement global puzzle reconstruction optimization.
- Support arbitrary puzzle shapes (not just square tiles).



THANK YOU